

**TEKNOLOGI IOT II**  
**LAPORAN PRAKTIKUM**



**DISUSUN OLEH:**

**NAMA : DEA ANGGRAENI**  
**NPM : 25782036**  
**KELAS :3B**  
**JURUSAN :TEKNOLOGI INFORMASI**  
**PRODI :TEKNOLOGI REKAYASA**  
**INTERNET**

**Dosen:**

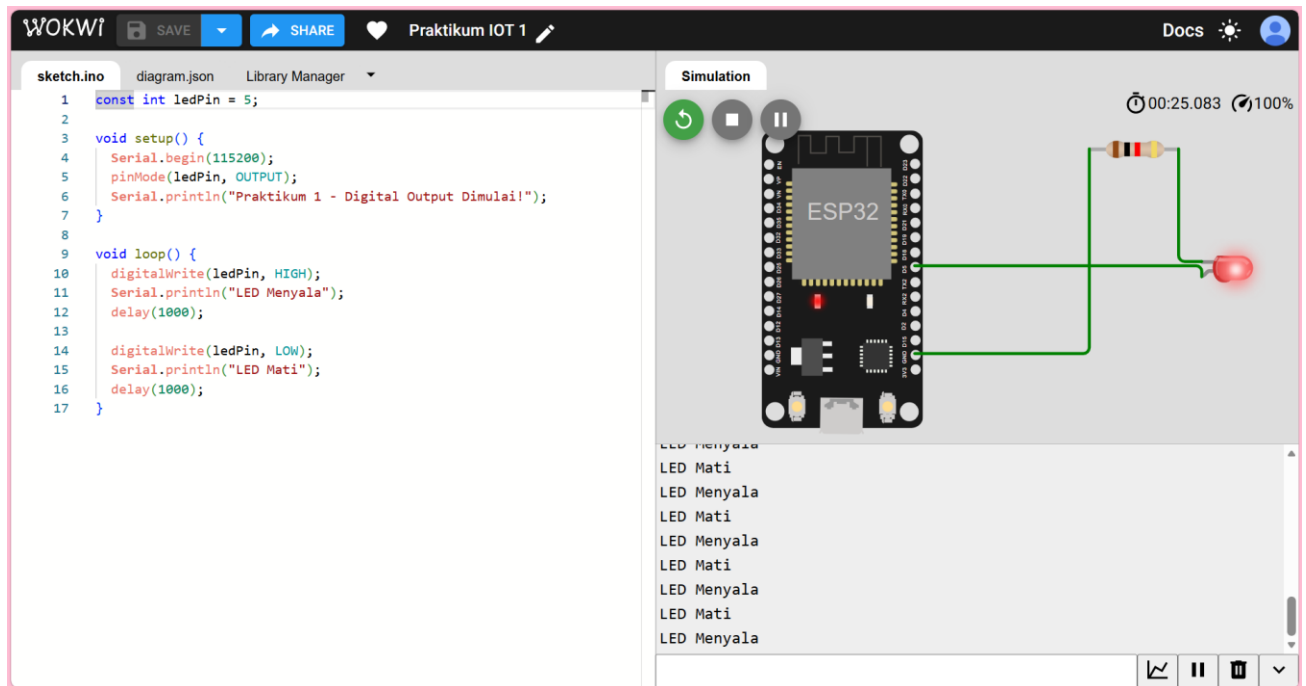
**Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

**Agus Ambarwari, SPd., M.Kom.**

**TEKNOLOGI REKAYASA INTERNET**  
**JURUSAN TEKNOLOGI INFORMASI**  
**POLITEKNIK NEGERI LAMPUNG**

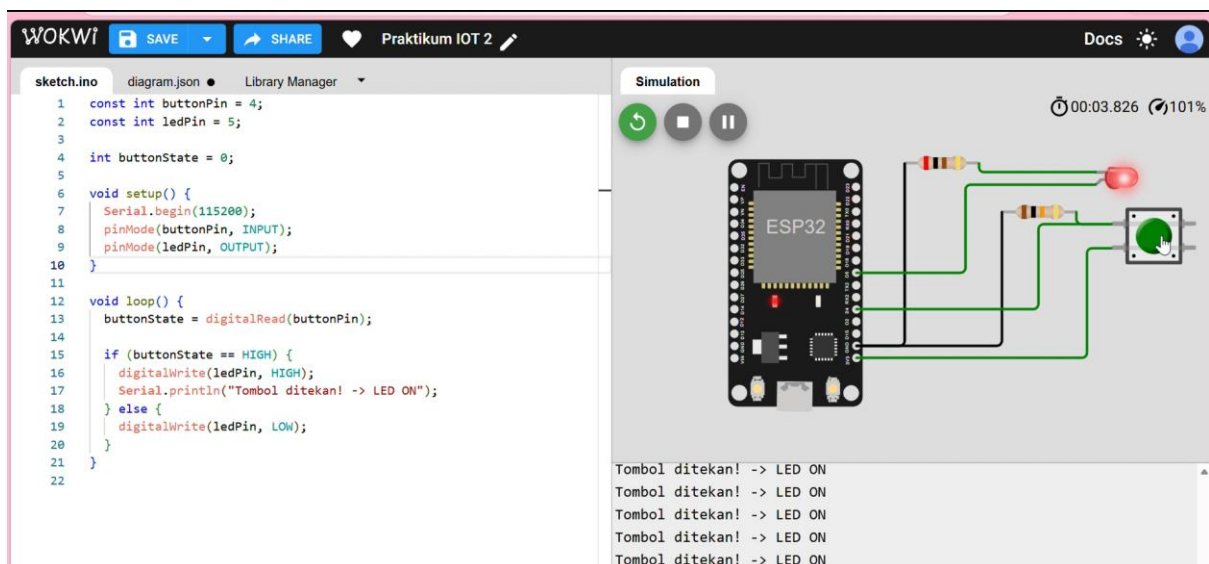
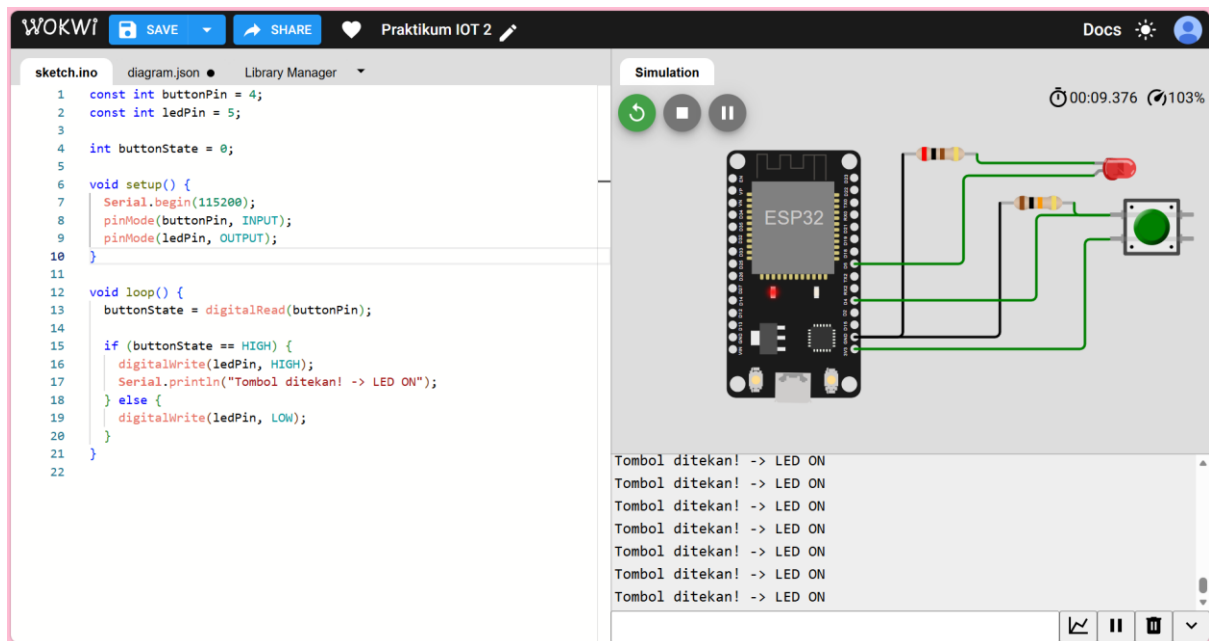
**2026/2027**

- **1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)**



Percobaan ini bertujuan untuk mengontrol nyala LED berbasis sinyal digital secara periodik. Tahap inisialisasi diawali dalam fungsi `setup()`, di mana pin terhubung dikonfigurasi sebagai *OUTPUT* dan saluran komunikasi serial diaktifkan dengan kecepatan *baud rate* 115200. Selanjutnya, alur kerja utama dieksekusi secara berulang pada fungsi `loop()`. Sinyal logika *HIGH* diberikan ke pin LED untuk mengalirkan tegangan sehingga LED menyala, bersamaan dengan pengiriman status "LED Menyala" ke Serial Monitor. Setelah jeda penundaan (*delay*) selama 1 detik, logika diganti menjadi *LOW* untuk memadamkan LED dan mengirim instruksi status "LED Mati". Siklus ini terus berulang secara konstan, membuktikan bahwa logika pemrograman telah terintegrasi dengan baik dan sistem perangkat keras merespons perintah secara tepat.

- 1.5.2 Praktikum 2: *Digital Input* (Membaca Push Button)



Praktikum ini menguji pembacaan input digital tombol *push button* untuk mengendalikan keluaran LED. Rangkaian memanfaatkan resistor *pull-down* 10 k $\Omega$  pada GPIO 4 agar sinyal tetap berada di kondisi *LOW* saat tombol dilepas. Melalui instruksi `digitalRead()`, mikrokontroler mendeteksi aliran arus 3.3V ketika tombol ditekan yang terbaca sebagai status *HIGH*. Kondisi tersebut memicu perintah `digitalWrite()` untuk menyalakan LED dan mengirimkan teks indikator ke Serial Monitor. Pengujian menunjukkan bahwa respons sistem berjalan secara presisi sesuai dengan status penekanan tombol.

## • 1.6 Latihan

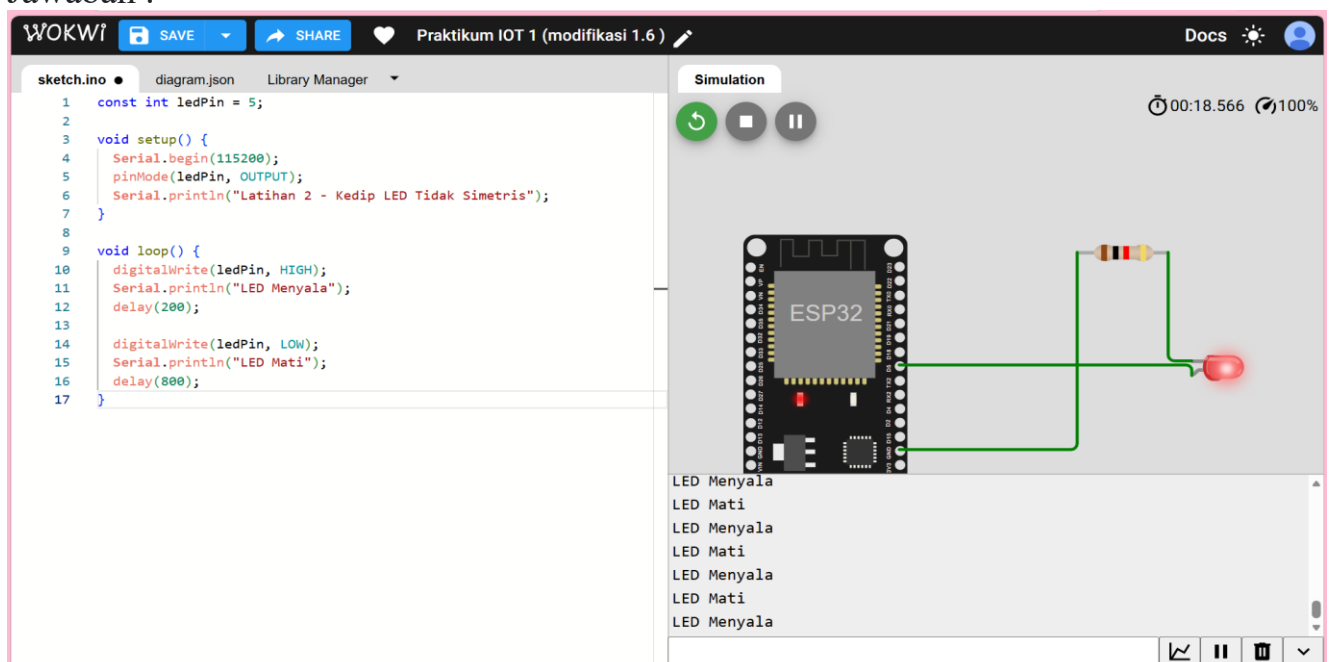
**Latihan 1 (Pemahaman Logika Pin):** Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawaban :

Pemasangan tombol *Active-High* pada pin D8 (GPIO 15) yang ditekan saat NodeMCU mulai dinyalakan akan mengakibatkan **kegagalan proses booting** (*boot failure*). Sesuai acuan tabel *pinout*, GPIO 15 harus berada pada logika *LOW* (terhubung ke GND) saat awal penyalan agar mikrokontroler dapat beroperasi dengan normal. Penekanan tombol pada sirkuit *pull-down* mengalirkan tegangan 3.3V (*HIGH*) ke pin tersebut secara paksa. Pelanggaran terhadap syarat *booting* ini membuat cip ESP8266 *stuck* dan tidak dapat mengeksekusi program utama. Sebagai solusinya, disarankan memakai pin yang aman untuk input eksternal seperti D1 (GPIO 5) atau D2 (GPIO 4).

**Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

Jawaban :



The screenshot shows the WOKWI simulation environment. On the left, the code editor displays the following C++ code for `sketch.ino`:

```
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Latihan 2 - Kedip LED Tidak Simetris");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(200);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(800);
17 }
```

On the right, the simulation window shows an ESP32 module connected to a red LED. The LED is currently lit. Below the simulation, a log window shows the following output:

```
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
```

The top of the interface shows the title bar "Praktikum IOT 1 (modifikasi 1.6)" and a timer at 00:18.566 with 100% battery.

Modifikasi program dilakukan pada durasi jeda agar nyala LED tidak simetris. Jika awal mulanya LED menyala dan padam masing-masing memakai delay(1000), sekarang nilainya diganti menjadi delay(200) ketika kondisi HIGH agar menyala sebentar, dan delay(800) ketika kondisi LOW agar padam lebih lama.

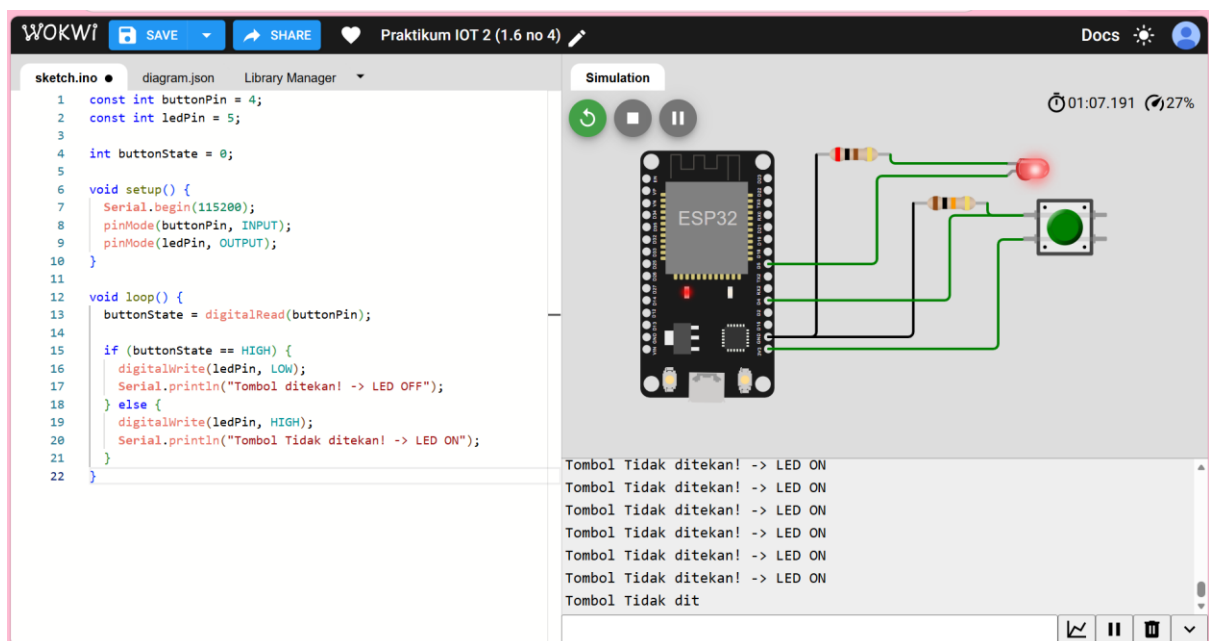
**Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawaban :

- **Apakah terjadi error kompilasi?** Tidak. Program tetap sukses terkompilasi (*Done uploading*) karena kesalahan pemasangan kaki LED merupakan kekeliruan sirkuit fisik (*hardware*), bukan kesalahan struktur sintaksis kode program.
- **Mengapa LED tidak menyala?** Berdasarkan sifat dasar dioda (LED) pada arus searah (DC), arus listrik hanya dapat mengalir satu arah dari Anoda ke Katoda (*Forward Bias*). Saat posisi kaki terbalik (Anoda ke GND dan Katoda ke D1), LED berada dalam kondisi prasikap balik (*Reverse Bias*). Hal ini membuat LED menyumbat aliran arus listrik, sehingga tidak ada arus yang mengalir dan LED tetap padam.

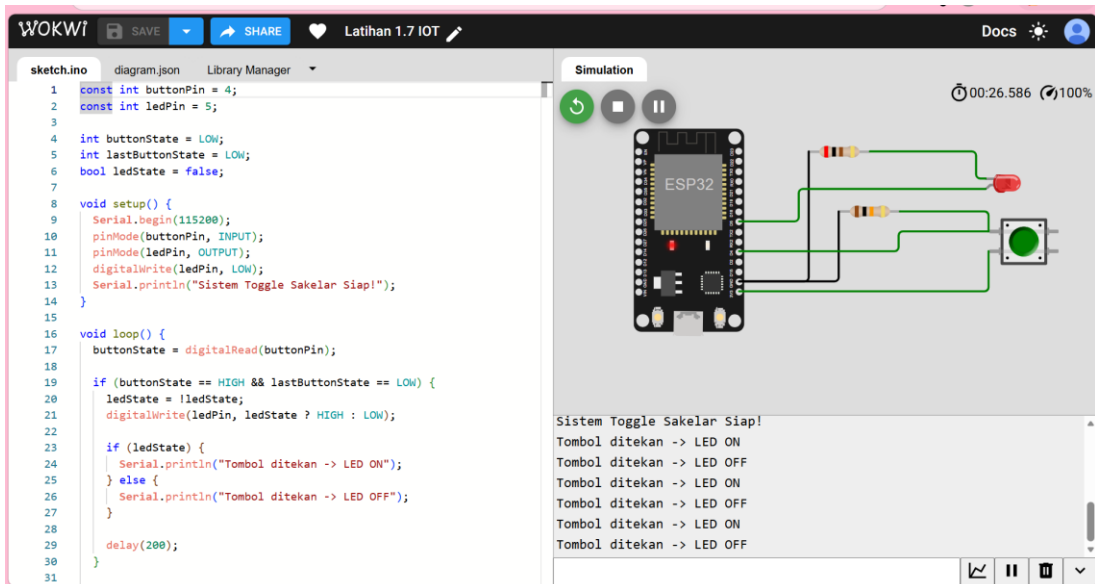
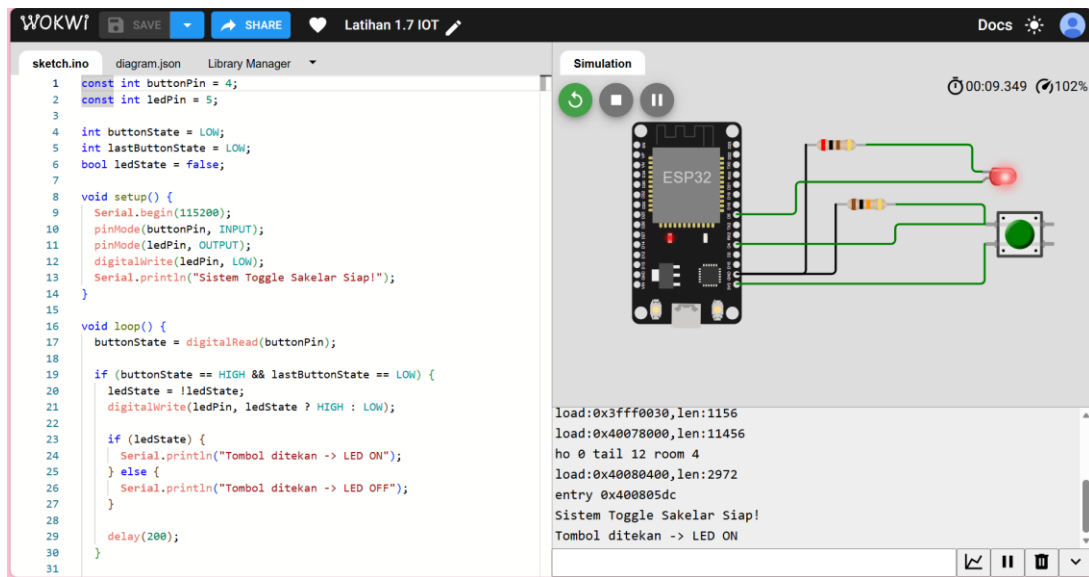
**Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

Jawaban :



Logika if-else dimodifikasi untuk membalik cara kerja LED. Karena memakai resistor *pull-down*, saat tombol ditekan pin membaca *HIGH*, lalu program mengirim instruksi LOW sehingga LED mati. Sebaliknya, saat tombol dilepas pin membaca *LOW*, lalu program mengirim instruksi HIGH sehingga LED menyala. Hasilnya, LED mati saat tombol ditekan dan menyala saat tombol dilepas.

## • 1.7 Tugas



Implementasi sakelar *latching* memanfaatkan variabel flag `ledState` berjenis boolean untuk memantau status aktif-padam LED secara berkelanjutan. Saat sirkuit pertama kali menerima suplai daya, `ledState` bernilai `false` sehingga LED memulai operasinya dari kondisi mati. Pemicuan tombol memanfaatkan deteksi tepi sinyal (*edge detection*) untuk mengenali transisi perubahan masukan. Setiap kali ada pemicuan tunggal yang valid, nilai logika `ledState` akan dibalik secara bergantian (ON menjadi OFF atau OFF menjadi ON). Instruksi `delay(200)` ditempatkan tepat setelah pembacaan sinyal guna mengeliminasi gangguan getaran mekanis (*debouncing*), menjamin bahwa satu kali tekanan sakelar hanya mengeksekusi satu kali perpindahan status LED tanpa fenomena kedipan acak (*flickering*).

